

Design Document: Airline Check-in System (ACS)

1. How many threads do I use? Specifying the task that I intended for each thread to perform.

I used a total of $1 + 5 + N$ threads, where N is the number of customers specified in the input file.

Main Thread: Used for reading the input file, initializing synchronization primitives (mutexes and condition variables), creating all clerk and customer threads, and joining them at the end to calculate statistics.

The Clerk Threads (5): These run in an infinite loop. Their task is to monitor the queues, dequeue customers based on priority (Business > Economy), signal the chosen customer to start, and wait for that customer to finish!

The Customer Threads (N): Each thread represents 1 traveler. They simulate arrival using `usleep()`, enter the queue, wait to be picked by a clerk, simulate the service duration using `usleep()` again, and finally signal the clerk that they have finished.

2. Do the threads work independently? Or is there an overall "controller" thread?

The threads operate independently without an overall controller thread beyond the main thread's initialization and joining. Clerks compete for customers via shared queues under mutual exclusion. Once the simulation starts, clerks look for work, and customers independently wait for service.

3. How many mutexes am I going to use? Specifying the operation that each mutex will guard.

I used two mutexes which are,

1. **queue_mutex:** This is the primary lock. It protects the critical sections involving the linked-list queues (`business_head`, `economy_head`), queue length counters, and the shared `customers_finished` counter. It makes sure that only 1 thread (clerk or customer) can modify the queue state at a time.

2. **stats_mutex:** This is a special lock used only when a customer finishes service. It protects the global variables (`total_waiting_time`, `count_all`, etc.) to prevent race conditions when multiple customers finish simultaneously and try to update the averages.

4. Will the main thread be idle? If not, what will it be doing?

The main thread is not idle. After creating the threads, it enters a "blocking wait" state using `pthread_join()`. It waits for every customer to finish. Once all customers are done, it sends a signal to any sleeping clerks to wake them up so they can terminate cleanly. Finally, it computes the average wait times and prints the results.

5. How do I represent customers? What type of data structure are used?

Customers are represented using a struct named `customer_t`. This structure stores arrival/service times, timing metadata (start/end times), and synchronization tools (a per-customer condition variable). To manage the queues, I used a **Linked List** implementation where each `customer_t` contains a next pointer.

6. How are you going to ensure that data structures in your program will not be modified concurrently?

I used **Mutual Exclusion** using `pthread_mutex_lock` and `pthread_mutex_unlock`. Every block of code that accesses or modifies a shared variable (like adding a customer to a list or decrementing a queue length) is wrapped in a mutex. This ensures that even if 5 clerks and 100 customers are running at the same time, access occurs only while holding the respective mutex, preventing races.

7. How many convars are you going to use?

I used a total of $1 + 5 + N$ condition variables:

- **customer_available (1):** Used by clerks to sleep when both queues are empty. Customers signal this when they arrive.
- **clerk_cond (5):** An array where each clerk has their own convar. A clerk waits on their specific convar while a customer is "self-serving" (sleeping).
- **cust->cond (N):** Each customer has a unique convar. They wait on this until a clerk "pick" them from the queue.

Associated Mutex: All convars are associated with `queue_mutex`.

Why: Because the condition being checked (e.g., "is the queue empty?" or "is `is_served` true?") It involves variables that are part of the shared state protected by that mutex.

Post-Wait Operation: Once `pthread_cond_wait` returns, the thread re-verifies the condition (using a `while` loop) to handle "spurious wakeups" and then proceeds to modify the shared state (like dequeuing or starting service).

8. Briefly sketch the overall algorithm you will use.

Main Thread:

1. Parse the input file and populate the global `customers []` array.
2. Initialize `queue_mutex`, `stats_mutex`, `customer_available`, the per-clerk condition variables, and each customer's `cond`.
3. Record the simulation at start time.
4. Create 5 clerk threads with IDs 1–5 running `clerk_thread_func`.
5. Create `num_customers` customer threads running `customer_thread_func`, each with a pointer to its `customer_t`.
6. Join all customer threads.
7. Lock `queue_mutex`, broadcast on `customer_available` to wake any sleeping clerks, unlock `queue_mutex`.
8. Join all clerk threads.
9. Compute average waiting times for all customers, business-class customers, and economy-class customers, then print them.
10. Destroy all mutexes and condition variables and exit.

Customer Thread

1. Sleep until arrival: `usleep(arrival_time * 100000);`
2. Print "A customer arrives" with its ID.
3. Lock `queue_mutex`.
4. Record `queue_enter_time` using the current simulation time.
5. Enqueue itself into the appropriate queue based on `class_type` (1 → business, 0 → economy), updating queue head/tail and length, and print the "enters a queue" message with queue ID and length.
6. Signal `customer_available` to wake a clerk if any are sleeping.
7. While `!is_served`, wait on its own condition variable:
`pthread_cond_wait(&cust->cond, &queue_mutex);`
8. Once woken, record `service_start_time`, compute waiting time, and print the "clerk starts serving" message.

9. Unlock `queue_mutex`.
10. Simulate service time with `usleep(service_time * 100000);`.
11. Lock `stats_mutex`, update global waiting time totals and counters, unlock `stats_mutex`.
12. Lock `queue_mutex`, increment `customers_finished`, signal `clerk_cond[served_by_clerk_id]` to wake the serving clerk, unlock `queue_mutex`.
13. Exit the thread.

Clerk Thread

1. Lock `queue_mutex`.
2. While `customers_finished < num_customers` or queues are non-empty:
 - While both queues are empty and `customers_finished < num_customers`, wait on `customer_available`:
`pthread_cond_wait(&customer_available, &queue_mutex);`
 - If all customers are finished and both queues are empty, unlock `queue_mutex` and exit.
 - Otherwise, dequeue a customer:
Prefer business queue if non-empty, otherwise use economy queue.
 - Set the customer's `served_by_clerk_id` and `is_served = 1`.
 - Signal the customer's condition variable (`cust->cond`) so the customer can start service.
 - Wait on the clerk's own condition variable:
`pthread_cond_wait(&clerk_cond[clerk_id], &queue_mutex);` to be notified when the customer finishes.
3. After being woken up by the customer, loop back to check queues and possibly serve another customer.